

Library A Guide

This document should include everything you need to use this library to complete the task. You are welcome to review it as needed.

How Interactive Formulas are Implemented

Step 1: Make a `FormulaContainer`

Create a container for the formula. This provides necessary context to children components like `VariableLabel` and `VariableHitArea`.

```
<FormulaContainer className="relative">
  {/* All other formula components go inside here */}
</FormulaContainer>
```

Step 2: Make a `Formula`

Use a `Formula` component to render a LaTeX math expression.

```
const latex = `\\\\cssId{var-y}{y} = \\\\cssId{var-x}{x} + 1
`;
<Formula latex={latex} />
```

Step 3. Add `VariableHitArea`s

Add hit areas for listening for interaction events on necessary symbols. For example:

```
<VariableHitArea
    elementId="var-x"
    isHovered={xHovered}
    onHover={setXHovered}
```

```
  value={x}
  onChange={setX} min={0} max={10} step={1}
/>
```

`value` / `onChange` / `min` / `max` / `step` configure how the value changes as the variable is dragged. The `elementId` must match what is in the `cssId` brackets from the formula latex.

Step 4. Add `VariableLabel`s

Add labels for variables. These are also interactive through dragging, like the hit areas.

```
<VariableLabel
    elementId="var-x"
    value={x}
    name="Input x"
  isHovered={xHovered}
  onHover={setXHovered}
  onChange={setX} min={0} max={10} step={1}
/>
```

The `elementId` must match what is in the `cssId` brackets from the formula latex.

Example Implementation (`y = x + 1`)

Here is a full example implementation of an interactive formula for y = x + 1, where `x` and `y` are both labeled, and changes to `x` lead to computing a derived value of `y`.

```
import { useState } from "react";
import { Formula, VariableLabel, VariableHitArea, FormulaCont
ainer } from "./ui";

function App() {
  const defaultX = 2;
  const [x, setX] = useState(defaultX);
  const [xHovered, setXHovered] = useState(false);
  const [yHovered, setYHovered] = useState(false);
```

```
  const y = x + 1;
  const latex = `\\\\cssId{var-y}{y} = \\\\cssId{var-x}{x} +
1`;
  return (
    <div className="flex justify-center p-12">
      <FormulaContainer className="relative">
        <div style={{ fontSize: "1.5rem" }}>
          <Formula latex={latex} />
          <VariableHitArea
            elementId="var-y"
            isHovered={yHovered}
            onHover={setYHovered}
          />
          <VariableHitArea
            elementId="var-x"
            isHovered={xHovered}
            onHover={setXHovered}
            value={x}
            onChange={setX} min={0} max={10} step={1}
          />
        </div>
        <div className="flex justify-center gap-8">
          <VariableLabel
            elementId="var-y"
            value={y}
            name="Output y"
            isHovered={yHovered}
            onHover={setYHovered}
          />
          <VariableLabel
            elementId="var-x"
            value={x}
            name="Input x"
            isHovered={xHovered}
            onHover={setXHovered}
            onChange={setX} min={0} max={10} step={1}
```

```
            />
          </div>
        </FormulaContainer>
      </div>
    );
  }
```

Shared Props

Below we provide a more comprehensive reference for the properties of the `VariableLabel` and `VariableHitArea` objects because they accept the same interaction props.

Prop	Type	Required	Description
`elementId`	`string`	Yes	Must match the `\\cssId{}` id in LaTeX.
`isHovered`	`boolean`	Yes	Whether this variable is currently hovered.
`onHover`	`(hovered: boolean) => void`	Yes	Callback to update hover state.
`value`	`number`	`VariableHitArea`: No `VariableLabel`: Yes	Current numeric value.
`name`	`string`	Label only	Name shown below the value.
`onChange`	`(value: number) => void`	No	Enables drag-to-change.
`min`	`number`	No	Minimum value when dragging.
`max`	`number`	No	Maximum value when dragging.
`step`	`number`	No	Value change per drag pixel. Defaults to `0.1`.